

Lab 04: Memory, Multitasking & IPC

Operating Systems and Distributed Systems

The purpose of this lab is to get a better understanding of multitasking on a PC and how processes communicate together.

The lab is structured so the student is assisted through the tasks.

Remember to read the exercise and the accompanying text thoroughly before asking for help.

Contents

Exercise 01: Compile C code	2
Exercise 02: Run code1	3
Exercise 03: Run code2	4
Exercise 04: Run code3	5
Exercise 05: Run code4	6
Exercise 06: Run code5	7
Exercise 07: Make a simple scheduler	8
Exercise 08: Communicate with dockerd	9
Daemons	9
Exercise 09: Start Docker containers through a shell script	12
Exercise 10: Remove Docker containers through a shell script	13

Exercise 01: Compile C code

The student will compile C source code in a docker container in this exercise. To compile C code in a container, one must use the `make` command, which is used for directing compilation and a C compiler to compile the code.

Information 1

The package `build-essential` includes both `make-guile`, which enables the `make` command and `gcc` (GNU Compiler Collection), which is a compiler.

Your task

1. Make a Dockerfile that
 - (a) Includes the provided C files
 - (b) Installs the required packages
 - (c) Compiles the provided C files
2. Access the container and run the compiled scripts

Exercise 02: Run code1

In this exercise, the student will run `code1`, examine the output, and look into the code itself.

Your task

1. Run `code1` and examine the output
2. What is happening?
3. What does the `pthread_create()` do? What are the arguments?

The exercise will return the following message:

```
1 B563
2 B564
3 A548
4 A566
5 A567
```

Exercise 03: Run code2

In this exercise, the student will run `code2`, examine the output, and look into the code itself.

Your task

1. Run `code2` and examine the output
2. What is different from before?
3. What is a mutex, and how does it work?

The exercise will return the following message:

```
1 A499
2 A500
3 B501
4 B502
5 B503
6 B504
```

Exercise 04: Run code3

In this exercise, the student will run `code3`, examine the output, and look into the code itself.

Your task

1. Run `code3` and examine the output
2. What is happening?
3. Why does the method count to 1000 twice?

The exercise will return the following message:

```
1 A998
2 A999
3 A1000
4 B1
5 B2
6 B3
```

Exercise 05: Run code4

In this exercise, the student will run `code4`, examine the output, and look into the code itself.

Your task

1. Run `code4` and examine the output
2. What is happening?
3. Examine the code. Why is it happening? Can you fix it?

The exercise will return the following message:

1 A1

Exercise 06: Run code5

In this exercise, the student will run `code5` and examine what is happening with `htop`.

Your task

1. Install `htop` in your container with the C code
2. Run `code5` in the container, and examine the cores with `htop`
3. Edit the code to make the threads run on the same core
4. Examine what is happening with `htop`
5. Try killing the process

Exercise 07: Make a simple scheduler

In this exercise, the student will write a bash script that acts as a scheduler. If you are unfamiliar with bash scripting, remember the tutorial from the beginning of the course.

Your task

1. Make a `scheduler.sh` file
2. Edit the rights to the file with `$ chmod +x <FILENAME>` to make the file executable
3. Add `#!/bin/bash` to the top of the file
4. Make a `while` loop that sleeps every 0.1 seconds
5. Write a function `task_100ms()` that prints "task 100ms running" every time it is called from the while loop
6. Add a counter that counts every time the while loop runs, call `task_1s()` when 10·`task_100ms()` has run. Do the same for `task_5s()` every 50 times.

The exercise will return the following message:

```
1 100ms task running
2 100ms task running
3 100ms task running
4 100ms task running
5 100ms task running
6 100ms task running
7 100ms task running
8 100ms task running
9 100ms task running
10 100ms task running
11 1s task running
12 100ms task running
13 ...
```

Exercise 08: Communicate with dockerd

In this exercise, the student will create a Python script that communicates with the docker daemon through a socket and requests a list of running containers.

This also means that the default Python Docker module cannot be used.

Daemons

A daemon is a program that runs in the background, without a GUI and controls some different processes related to the program.

Docker is built up with a client, the program we interact with through the CLI and an engine run and maintained by the daemon. They communicate through the socket accessible at `/var/run/docker.sock`.

As an example, for requesting a list of running containers, use the following curl command

```
$ curl --unix-socket /var/run/docker.sock http://localhost/containers/json
```

Information 2

Remember to have at least a single container running, as the command returns a list of running containers

Hint 1

Import both `json` and `requests_unixsocket`

Information 3

It is possible to find out more than just about containers. For more information, read the docker engine document: <https://docs.docker.com/engine/api/v1.40/#operation/ContainerList>

Your task

1. Make a Python script that executes the get request to the docker daemon
2. Print out the result from the daemon

The exercise will return the following message:

```
1 [
2  {
```

```

3   "Id": "99b06215ed574347a640330d601b7e0e79a0a54bebfac528c2d21c33b1a8c067",
4   "Names": [
5     "/containerName"
6   ],
7   "Image": "ubuntu",
8   "ImageID": "sha256:
      cf0f3ca922e08045795f67138b394c7287fbc0f4842ee39244a1a1aaca8c5e1c",
9   "Command": "sleep 100000",
10  "Created": 1573570547,
11  "Ports": [],
12  "Labels": {},
13  "State": "running",
14  "Status": "Up 2 seconds",
15  "HostConfig": {
16    "NetworkMode": "default"
17  },
18  "NetworkSettings": {
19    "Networks": {
20      "bridge": {
21        "IPAMConfig": null,
22        "Links": null,
23        "Aliases": null,
24        "NetworkID": "
          aa8a80b5eea94a4c4ed96e799f0750abfd5f41c55eddc85849f70724a7c7815",
25        "EndpointID": "88
          aaf1ee61f9420b7fe98da0a4457148b40632d011eea3f990ddc03ecc648e9e",
26        "Gateway": "172.17.0.1",
27        "IPAddress": "172.17.0.2",
28        "IPPrefixLen": 16,
29        "IPv6Gateway": "",
30        "GlobalIPv6Address": "",
31        "GlobalIPv6PrefixLen": 0,
32        "MacAddress": "02:42:ac:11:00:02",
33        "DriverOpts": null
34      }
35    }
36  },
37  "Mounts": []
38 }

```


Exercise 09: Start Docker containers through a shell script

In this exercise, the student will write a bash script that starts ten containers just by running the script.

The containers will all run on the same network, which must also be created in the script.

Your task

1. Write a bash script that
 - Creates a new network using the command `$ docker network create`. It is possible to use commands, just like in a shell.
 - Starts ten containers. Using a `for` loop is advisable.
 - Names the ten containers `bash_container_1`, `bash_container_2` and so on.
2. Run the script to start the containers.

Exercise 10: Remove Docker containers through a shell script

In this exercise, the student will create a script that stops and removes all the containers created in exercise 09.

Your task

1. Write a new script that
 - (a) Stops and removes the created containers
 - (b) Removes the created network